MODULE 23 · DOMAIN 3 - DEPLOYMENT AND ORCHESTRATION

Scaling endpoints

Target tracking, step, scheduled, and scaling all the way to zero.

MODULE 23 of 43 DOMAIN 3 - Deploy WEIGHT 24%

admin@sthithapragna.com

Too few instances and requests fail. Too many and you pay for idle capacity all night.

What auto scaling actually does

- **Application Auto Scaling** adjusts the instance count behind an endpoint variant.
- You register the variant as a **scalable target**, with a minimum and maximum capacity.
- You attach one or more **scaling policies** that decide when the count changes.
- It is **horizontal** scaling: more instances, not bigger ones. Instance type is fixed.
- Changing instance type means a new endpoint configuration, from module 21.

> ON THE EXAM
>
> Auto scaling adds instances. It never changes the instance type, so a badly sized instance stays badly sized at any count.

Three ways to decide

POLICY	HOW IT BEHAVES
Target tracking	Keeps a metric at a target value. AWS recommends starting here
Step scaling	Changes capacity by defined steps as a CloudWatch alarm breaches thresholds
Scheduled scaling	Changes capacity at a time you specify, regardless of the metric

ON THE EXAM

They combine. AWS suggests target tracking as the base, with step scaling added for a more aggressive response at high load.

Target tracking

- You pick a metric and a target value. Auto scaling holds the metric near that value.
- It is the simplest to reason about, and AWS **strongly recommends** it as the starting point.
- It scales out and in automatically, without you defining thresholds.
- CloudWatch alarms are created for you behind the scenes.
- One policy covers both directions, which is why it is less work to maintain.

> ON THE EXAM
>
> Target tracking is a thermostat. You set the temperature; it works out how hard to run the heating.

The recommended metrics

- **SageMakerVariantInvocationsPerInstance**: the average invocations per minute per instance.
- **CPUUtilization**: the sum of work handled by the CPU across instances.
- AWS names both of these as the recommended metrics for target tracking.
- **Custom metrics** work too, provided the metric **changes in proportion to capacity**.
- That proportionality requirement is the catch. A metric that does not track load cannot drive scaling.

> ON THE EXAM
>
> Invocations per instance is the usual answer. CPU utilisation suits models where request cost varies a lot.

Choosing a target value

- **Too high** and instances saturate before scaling completes. Latency spikes during the gap.
- **Too low** and you run more instances than needed, permanently.
- Derive it from a **load test**: find where latency degrades, then target comfortably below it.
- Inference Recommender from module 22 gives you that curve without hand-rolling the test.
- Leave headroom for the time it takes a new instance to become useful.

> ON THE EXAM
>
> The target is not the maximum an instance can handle. It is the level at which you want more capacity already arriving.

Load testing before you configure

- Push a single instance until latency degrades, and record where that happens.
- That gives you the **capacity of one instance**, which every other number depends on.
- **Inference Recommender** runs this for you across instance types. Module 22.
- Then set the target below that point, leaving room for the scale-out lag.
- A policy configured without this is a guess wearing the clothes of a measurement.

> ON THE EXAM
>
> Every number in a scaling policy derives from one measurement: how much traffic can a single instance actually take?

Metrics that tell you scaling is healthy

METRIC	WHAT IT TELLS YOU
`Instance count over time`	Does it track demand, or oscillate, or never move?
`ModelLatency`	Time inside the container. Rising means saturation
`OverheadLatency`	SageMaker AI overhead. Rising can mean provisioning churn
`Invocation4XXErrors / 5XXErrors`	Failures. Spikes at scale events are a warning
`InvocationsPerInstance`	The scaling signal itself. Compare to your target

ON THE EXAM

Plot instance count against latency. If latency spikes just before instance count rises, your policy is reacting too late.

Step scaling

- A CloudWatch alarm breaches, and capacity changes by an amount you defined for that breach.
- Multiple steps let you react **proportionally**: a little over target adds one, far over adds five.
- Useful when demand can jump sharply and a proportional response is too slow.
- AWS suggests it as an **additional** policy alongside target tracking, not a replacement.
- More configuration to get right, and more to maintain.

> ON THE EXAM
>
> Step scaling is how you say 'if it gets really bad, add a lot at once' rather than converging gradually.

Scheduled scaling

- Change minimum and maximum capacity **at a time you specify**, on a schedule.
- It scales **ahead of** demand rather than in response to it, which reactive policies cannot do.
- Right when the pattern is known: a daily peak, a weekly batch, a month-end surge.
- Also how you take a development endpoint down overnight and bring it back at 8am.
- It ignores actual demand, so pair it with target tracking for the unexpected.

> ON THE EXAM
>
> Reactive scaling is always late by definition. If you know the peak is at 9am, scheduled scaling is the only way to be ready at 9am.

Cooldowns and capacity bounds

- **Scale-out cooldown**: how long to wait before scaling out again, so you do not overshoot.
- **Scale-in cooldown**: usually longer, so a brief lull does not remove capacity you need back.
- Cooldowns too short cause **thrashing**: capacity oscillating and never settling.
- **Minimum capacity** is your availability floor. Two instances is the practical production minimum.
- **Maximum capacity** caps cost, and also caps your ability to absorb a spike. Set it deliberately.

> ON THE EXAM
>
> Scale in slowly, scale out quickly. The cost of being briefly over-provisioned is far lower than the cost of being under.

DisableScaleIn

- A target tracking policy can be configured to **scale out but never scale in**.
- Legitimate when scaling in is risky: long-lived connections, or a slow-to-warm cache.
- **Capacity then only ever grows**, so cost ratchets up and never comes down.
- To reduce capacity afterwards you adjust the minimum manually.
- It is a common and genuinely puzzling cause of an endpoint bill that will not fall.

> ON THE EXAM
>
> If an endpoint scaled out during a spike three weeks ago and is still that size, check this setting first.

A real-time endpoint can now scale **all the way to zero**. The conditions are specific.

What scale-to-zero needs

- The endpoint must be **in service**.
- **It must host one or more inference components.** This is a hard requirement, not a recommendation.
- The production variant's **ManagedInstanceScaling** must have **MinInstanceCount set to 0**.
- Each inference component is registered as a scalable target with **minimum capacity 0**.
- The scalable dimension is the component's **DesiredCopyCount**, not an instance count.

> ON THE EXAM
>
> Module 21's inference components are the prerequisite. An endpoint without them cannot scale to zero at all.

How an endpoint at zero wakes

- You create a **step scaling policy** and assign it to a **CloudWatch alarm**.
- The endpoint provisions an instance soon after it receives a request it cannot serve.
- **Provisioning takes several minutes.** Not seconds.
- **During that time, invocations produce an error.** They do not queue and they do not wait.
- Those failures appear in CloudWatch as **NoCapacityInvocationFailures**.

> ON THE EXAM
>
> This is the critical difference from serverless. A serverless cold start is a slow response. Scale-to-zero from nothing is a failed one.

Where scale-to-zero fits, and where it does not

SITUATION	SUITABLE?
Predictable traffic with known quiet periods	Yes. AWS names this case
Sporadic workloads with long gaps	Yes, if callers can retry
Development and test endpoints	Yes. AWS names this case too
Customer-facing synchronous API	No. The first request after idle fails
Low but constant background traffic	No. It never gets to scale in

ON THE EXAM

Pair it with scheduled scaling when the pattern is known: scale out before the quiet period ends, and nothing ever fails.

Metrics for inference component scaling

- Components scale on **SageMakerInferenceComponentConcurrentRequestsPerCopy**.
- A **high-resolution** variant of that metric exists, for faster reaction.
- The target is **concurrent requests per copy**, which suits generation workloads far better than invocation count.
- Each component scales independently, so a busy model gets copies without affecting its neighbours.
- This is the finer-grained scaling module 21 described, made concrete.

> ON THE EXAM
>
> Invocations per minute is a poor signal for generation, where one request can occupy an instance for a minute. Concurrency is better.

Scaling GPU workloads

- **GPU instances start slowly.** Provisioning plus a large model load can be many minutes.
- So reactive scaling is often **too late**. Scheduled scaling and headroom matter more here.
- **Compilation and fast model loading** from module 22 directly reduce that scale-out time.
- GPU quotas from module 14 cap your maximum, often lower than you expect.
- GPU utilisation is a better scaling signal than invocation count for deep learning models.

> ON THE EXAM
>
> The bigger the model, the later reactive scaling arrives. That is why module 22's load-time optimisations are a scaling lever.

Scaling asynchronous and serverless endpoints

- **Asynchronous** endpoints scale on **ApproximateBacklogSizePerInstance**, the queue depth per instance.
- Queue depth is the natural signal there, because requests wait rather than failing.
- Async scales to zero natively, with no inference component requirement.
- **Serverless** scales itself. You set maximum concurrency rather than instance counts.
- Provisioned Concurrency on serverless can itself be auto scaled, from module 20.

ON THE EXAM

Async is the gentler scale-to-zero: a request to a sleeping async endpoint queues, where one to a sleeping real-time endpoint fails.

Which scaling approach

WHAT THE SCENARIO SAYS	THE ANSWER
General variable traffic	Target tracking on invocations per instance
Request cost varies a lot per call	Target tracking on CPU or GPU utilisation
A known daily or weekly peak	Scheduled scaling, plus target tracking
Sharp unpredictable spikes	Target tracking plus step scaling
Dev endpoints idle overnight	Scale to zero, or scheduled scaling to zero
Generation workload, several models	Inference components on concurrent requests per copy
Cost high after a past spike	Check DisableScaleIn

ON THE EXAM

Known pattern means scheduled. Unknown pattern means target tracking. Sharp spikes mean adding step scaling on top.

Testing a scaling policy

- Generate load that crosses the threshold, and watch what the policy actually does.
- **Time the scale-out**: from threshold breach to the new instance serving traffic.
- That number is what you must leave headroom for. Guessing it is how morning spikes happen.
- Test scale-in too, and confirm capacity actually comes back down.
- An untested policy is a policy you will first observe during an incident.

> ON THE EXAM
>
> Scale-out time is the single number that determines whether reactive scaling can work for you at all.

When auto scaling is not the answer

- **A spike faster than provisioning** cannot be absorbed by scaling. Only headroom absorbs it.
- **Queue instead:** asynchronous inference turns a spike into a backlog rather than failures.
- **Shed load deliberately** with throttling, so the important traffic survives.
- **Or size for the peak** if the peak is most of the day. Auto scaling suits variable load, not constant load.
- A known event, such as a product launch, is a capacity request rather than a scaling policy.

> ON THE EXAM
>
> Auto scaling handles variability. It does not handle instantaneous spikes, and it does not create quota you do not have.

What scaling saves, and costs

SETTING	EFFECT ON COST
Minimum capacity	Your floor. This is what you pay all night, every night
Maximum capacity	Your ceiling. Caps cost, and caps spike absorption
Target value	Lower target means more instances held permanently
Scale-in cooldown	Longer means capacity is held after demand falls
Scale to zero	Removes the floor entirely, at the cost of failed first requests
DisableScaleIn	Removes scale-in entirely. Cost only goes up

ON THE EXAM

Minimum capacity is the single most important cost number on an endpoint, because it is the one that never goes away.

Scaling does not fix the wrong instance

- If a single request saturates one instance, scaling gives you many saturated instances.
- Check **utilisation at low load** first, using module 22's table.
- An undersized instance scales expensively. A right-sized one scales efficiently.
- Conversely, an oversized instance means auto scaling rarely triggers and you pay for headroom.
- Size the instance in module 22, then scale it here. That order matters.

> ON THE EXAM
>
> Auto scaling multiplies whatever instance decision you made. It multiplies a good one and a bad one equally.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Latency spikes at the start of every peak	Reactive scaling arriving late. Use scheduled scaling
Capacity oscillating up and down	Cooldowns too short. Thrashing
Cost never falls after a spike	DisableScaleIn is set
NoCapacityInvocationFailures in CloudWatch	Endpoint at zero. Requests arrived before provisioning finished
Scale-to-zero will not enable	No inference components on the endpoint
Scaling stops below the policy maximum	Instance quota reached, not the policy limit
Adding instances did not improve throughput	Bottleneck is elsewhere. Module 22's utilisation table

ON THE EXAM

Row four is the one to recognise on sight. It means the endpoint was asleep and a caller paid for it.

Reactive scaling is always **late**. Where the pattern is known, schedule it instead.

> A retail recommendation endpoint sees traffic rise sharply every weekday at 09:00 as stores open, and fall again after 18:00. The pattern has been consistent for two years.
>
> Target tracking is configured on invocations per instance. Every morning, latency spikes for six to eight minutes before settling. Customers notice.

What should they change?

A. Lower the target value, so scaling triggers sooner

B. Add step scaling with aggressive steps at high invocation counts

C. Add scheduled scaling to raise minimum capacity before 09:00

D. Shorten the scale-out cooldown so instances are added faster

◆ PAUSE AND DECIDE

What should they change?

A Lower the target value, so scaling triggers sooner

Shortens the gap and does not close it, because provisioning still starts after the traffic arrives. It also pays for more instances all night.

B Add step scaling with aggressive steps at high invocation counts

Reacts harder once the spike has been detected. The instances still take minutes to arrive, so the first minutes remain slow.

C Add scheduled scaling to raise minimum capacity before 09:00

The pattern is known and consistent for two years. Scheduled scaling provisions ahead of demand, which is the only thing a reactive policy cannot do.

D Shorten the scale-out cooldown so instances are added faster

Cooldown governs the wait between scaling actions, not provisioning time. Shortening it risks overshoot and thrashing.

THE SIGNAL

'Consistent for two years' is the giveaway. A known pattern does not need detecting; it needs anticipating.

A team runs eight development endpoints for model experimentation. They are used sporadically during working hours and not at all overnight or at weekends. Nobody is a customer; the users are the team itself.

The endpoints are deployed using inference components. Finance has asked why the development account bills continuously.

What should they configure?

A Scale to zero, with MinInstanceCount 0 and each component registered at minimum capacity 0

B Target tracking with a very high target value, so few instances are needed

C Move all eight models onto one multi-model endpoint

D Switch all eight to asynchronous inference endpoints

◆ PAUSE AND DECIDE

What should they configure?

A Scale to zero, with MinInstanceCount 0 and each component registered at minimum capacity 0

Development and testing is a case AWS names for scale-to-zero, the endpoints already host inference components, and the users can tolerate a wait.

B Target tracking with a very high target value, so few instances are needed

Target tracking cannot go below the minimum capacity, which is at least one. It reduces the bill without removing the overnight floor.

C Move all eight models onto one multi-model endpoint

Reduces eight bills to one, and still bills continuously. It also assumes all eight share a framework, which is not stated.

D Switch all eight to asynchronous inference endpoints

Async does scale to zero, and it changes every caller to a polling or SNS pattern for an interactive development workflow.

THE SIGNAL

The prerequisite is already satisfied: these endpoints host inference components, without which scale-to-zero is not available at all.

> An endpoint scaled from 2 instances to 11 during a major incident three weeks ago. Traffic returned to normal the following day.
>
> The endpoint is still running 11 instances. Target tracking is configured and the metric has been far below target for three weeks. No manual changes were made.

What is the most likely cause?

A. The scale-in cooldown is set to an unusually long value

B. DisableScaleIn is set to true on the target tracking policy

C. Minimum capacity was raised to 11 automatically during the incident

D. The target value is set too low, so 11 instances are genuinely needed

◆ PAUSE AND DECIDE

What is the most likely cause?

A. The scale-in cooldown is set to an unusually long value

A cooldown delays the next scaling action. No plausible cooldown value holds capacity for three weeks.

B. DisableScaleIn is set to true on the target tracking policy

It scales out and never in, so capacity ratchets up after every spike and stays there. Reducing it requires a manual change to minimum capacity.

C. Minimum capacity was raised to 11 automatically during the incident

Auto scaling adjusts desired capacity within the bounds you set. It does not raise the minimum itself.

D. The target value is set too low, so 11 instances are genuinely needed

The scenario states the metric has been far below target for three weeks. If 11 were needed, the metric would be at target, not below it.

THE SIGNAL

The metric being below target for three weeks is the evidence. Scale-in should have fired repeatedly, and something prevented it.

> A customer-facing product search API is served by a real-time endpoint using inference components. Traffic is low overnight but never quite zero, with a handful of requests an hour from other time zones.
>
> To cut cost, an engineer proposes enabling scale-to-zero with MinInstanceCount 0, arguing that overnight traffic is negligible.

How should you respond?

A Agree. Negligible overnight traffic is exactly the case for scale-to-zero

B Agree, but add a step scaling policy so wake-up is faster

C Reject it. Scale-to-zero requires inference components, which this endpoint lacks

D Reject it. Overnight requests would fail during the several minutes of provisioning

◆ PAUSE AND DECIDE

How should you respond?

A. Agree. Negligible overnight traffic is exactly the case for scale-to-zero

Negligible is not zero. Each overnight request would arrive at a sleeping endpoint and fail, and these are customers.

B. Agree, but add a step scaling policy so wake-up is faster

The step policy and alarm are already part of how scale-to-zero works. Provisioning still takes several minutes.

C. Reject it. Scale-to-zero requires inference components, which this endpoint lacks

Right conclusion, wrong reason. The scenario states the endpoint already uses inference components, so the prerequisite is met.

D. Reject it. Overnight requests would fail during the several minutes of provisioning

Requests to an endpoint at zero produce errors, not slow responses, and appear as NoCapacityInvocationFailures. These are customers, so failures are not acceptable.

THE SIGNAL

This is the distinction from serverless. A serverless cold start delays a response by seconds; waking from zero fails the request outright.

What to take from this module

- **Target tracking is the default**, on invocations per instance or CPU utilisation.
- **Known pattern means scheduled scaling.** Reactive policies are late by construction.
- **Scale-to-zero needs inference components** and MinInstanceCount 0. Both, not either.
- **Waking from zero takes minutes, and requests fail meanwhile.** Not a customer-facing option.
- **Minimum capacity is the number that never goes away.** It is what you pay every night.

ON THE EXAM

Module 24 covers changing what is deployed: blue/green, canary, shadow testing and rollback.